Software Engineering: Algorithms and Data Structures
Example 3
MARK: 54%

```
#include <iostream>
#include <string>

using namespace std;

struct Node{
	string item;
	Node* next;
};

void insmlistend(Node* &hdList, string name){
	Node* newNode = new Node;
	newNode->item = name;
	newNode->next = NULL;

	if(hdList == NULL){
		hdList = newNode;
	}
	else{
		insmlistend(hdList->next, name);
	}
}

void printmlist(Node* hdList){
	while(hdList!=NULL){
		cout << hdList->item << endl;
		hdList = hdList->next;
	}
}

/// The function does not work, as it always gives that the list is sorted
bool ismlistsorted(Node* hdList){
	bool sorted = true;
	string lastname;
	/// there is no protection againsth an empty list
	if(hdList->next!=NULL){
		lastname = hdList->item;
		ismlistsorted(hdList->next);  /// the function returns a values which is not
used
		if(hdList->item < lastname){
			sorted = false;
		}
```

```
        }
        return sorted;
}

/// The function does not work, and it stucks in an infinite loop
void swapfirstlastmlist(Node* &hdList){
        Node* firstHd = hdList;
        Node* lastPtr = hdList;
        while(hdList!=NULL){
                if(hdList->next == NULL){               /// the pointer update here result in a
structure with a loop (that is why your program stucks here)
                        hdList->next = firstHd->next;
                        firstHd->next = NULL;
                        lastPtr->next = firstHd;
                        firstHd = hdList;
                }
                lastPtr = hdList;
                hdList = hdList->next;

        }
        hdList = firstHd;
}


int main(){
        Node* hdList = NULL;
        string name;
        while(name!="STOP"){
                cout << "enter element:" << endl;
                cin >> name;
                if(name != "STOP"){
                        insmlistend(hdList, name);
                }
        }

        cout << "printing the list:" << endl;
        printmlist(hdList);
        cout << endl;
        if(ismlistsorted(hdList)==true){
                cout << "the list is sorted" << endl;
        }
        else{
                cout << "the list is not sorted" << endl;
        }

        swapfirstlastmlist(hdList);
        cout << "printing the list after swapping the first and the last node: "<< endl;
```

```
        printmlist(hdList);

        return 0;
}
```